

Documento maestro por fases — Backend (V1) para proyectos freelance

Este lienzo es tu **plantilla universal** para construir backends CRUD (tipo tienda, inventario, ventas, pedidos) con una ingeniería sólida.

Se basa en lo que venimos trabajando: - **Levantamiento de información** y narrativa del negocio - Diseño por módulos (NestJS Modules) - Base de datos con **inventario por movimientos** - Reglas clave: **no borrar, ventas cerradas no se editan, correcciones con devoluciones** - Contrato API claro (REST + Swagger) - Onboarding + Scaffolding profesional (repo reproducible)

Meta: que 1 sola persona pueda mantener el proyecto sin morir, y que el sistema pueda crecer sin volverse un “monstruo”.

Glosario mínimo (para no perderse)\$1---

Reglas globales (para que el documento sea consistente)

Estas reglas aplican a **todas** las fases. Te evitan el 80% de los problemas en freelance.

Precauciones globales (siempre)

- **Nada es obvio:** entidades, cardinalidades, flujos y excepciones se preguntan y se documentan.
- **No adivines reglas de negocio:** si el cliente no lo define, queda como **decisión abierta**.
- **Evita la magia invisible:** cualquier cosa “automática” (triggers, cascadas raras, jobs ocultos) se documenta y se testea.
- **Protege el historial:** no borrar; preferir desactivar y registrar movimientos/cambios.
- **Una fase cerrada antes de la siguiente:** si se reabre una fase, se anota como cambio (para controlar scope).

Observaciones globales (siempre)

- Tu mayor ahorro de tiempo viene de **decisiones correctas**, no de teclear archivos.
- El “monstruo” aparece cuando no hay límites: **módulos claros + contrato API + reglas**.
- Cuando el sistema crece, lo que más duele es:
 - estados mal definidos (pedido/venta)
 - inventario inconsistente
 - cambios sin trazabilidad

Entregables globales (siempre)

- **Documentos** (docs /): levantamiento, API, RBAC, decisiones, DB.
 - **Repo reproducible:** docker compose, env example, scripts.
 - **Calidad mínima:** lint + format + tests críticos.
-

Mapa de fases (vista rápida)

Fase -1 — Acuerdo y expectativa (freelance) 1) Fase 0 — Alineación y alcance (Scope) 2) Fase 1 — Levantamiento de información (Discovery) 3) Fase 2 — Modelo de dominio (Domain design) 4) Fase 3 — Modelo de datos (DB design + normalización) 5) Fase 4 — Contrato API (REST + DTOs + Swagger) 6) Fase 5 — Arquitectura (módulos + capas + decisiones) 7) Fase 6 — Onboarding + Scaffolding (repo profesional) 8) Fase 7 — Desarrollo (implementación por módulos) 9) Fase 8 — Testing (unit + integration + e2e) 10) Fase 9 — Calidad y hardening (seguridad + performance + DX) 11) Fase 10 — Release/Deploy (entrega/producción) 12) Fase 11 — Operación y mantenimiento (observabilidad + mejoras)

Fase -1 — Acuerdo y expectativa (Freelance)

Objetivo

Alinear con el cliente **expectativa, precio, tiempo, responsabilidades**.

Pasos

1. Definir entrega: ¿qué es “terminado” en V1?
2. Definir qué incluye mantenimiento: bugs vs nuevas features.
3. Definir entorno del cliente:
4. ¿tiene hosting?
5. ¿quién paga dominio/servidor?
6. Definir flujo de cambios:
7. “una solicitud = un ticket”
8. prioridades

Precauciones

- No prometer V2/V3 dentro del costo de V1.
- No empezar a programar sin un alcance mínimo firmado.

Observaciones

- Esta fase te ahorra problemas legales y de expectativas.

Salidas

- Lista de módulos V1
 - Lista de no-alcance
 - Orden de implementación
-

Fase 0 — Alineación y alcance (Scope)

Objetivo

Definir **qué entra** y **qué no entra** en la V1 (mínimo vendible).

Entradas

- Tipo de negocio (tienda, barbería, bodega, etc.)
- Roles (quién usa el sistema)

Pasos

1. Definir el “core” del negocio (lo esencial).
2. Definir módulos V1:
3. Auth + Users/Roles
4. Catálogo
5. Operación (ventas/compras/inventario)
6. Pedidos/envíos (si aplica)
7. Reportes básicos
8. Definir no-alcance claro:
9. multi-bodega
10. promociones complejas
11. facturación oficial
12. contabilidad completa

Precauciones

- La V1 muere si quieres hacer TODO desde el día 1.
- Si el cliente cambia de opinión cada semana, pide decisiones por escrito.

Observaciones

- Una V1 buena es **usable y vendible**, no perfecta.

Salidas

- Lista de features V1
 - Lista de features futuras (V2/V3)
-

Fase 1 — Levantamiento de información (Discovery)

Objetivo

Convertir “lo que el cliente cuenta” en: - **casos de uso** - **reglas de negocio** - **datos necesarios**

Entradas

- Conversación con el cliente
- Documentos, Excel, facturas, WhatsApp, etc.

Pasos

1. Escribir narrativa del negocio (prosa):
2. qué vende, cómo vende, cómo cobra, cómo repone stock
3. Extraer actores:
4. ADMIN (configura)
5. VENDEDOR (vende)
6. BODEGA (compra/stock)
7. Extraer casos de uso:
8. Registrar venta
9. Registrar compra
10. Ajustar inventario
11. Pedido WhatsApp
12. Reportes
13. Extraer reglas:
14. **no borrar → desactivar**
15. **venta cerrada no se edita**
16. **inventario por movimientos**
17. Hacer lista de decisiones NO obvias:
18. ¿se permite venta sin cliente?
19. ¿pagos parciales?
20. ¿código SKU único?

Precauciones

- No “adivines” cardinalidades: todo se pregunta.
- Evita cambiar reglas en medio del desarrollo: primero cierra la fase.

Observaciones

- Este documento es el que te salva cuando el cliente dice: “yo nunca pedí eso” o “eso estaba implícito”.

Salidas

- docs/LEVANTAMIENTO.md

- Lista de preguntas y decisiones
-

Fase 2 — Modelo de dominio (Domain design)

Objetivo

Definir el **mundo del sistema** con conceptos claros: qué existe, cómo se relaciona y qué estados tiene.

Entradas

- Levantamiento ya escrito

Pasos

1. Definir entidades principales:
2. Category / Product / SKU
3. Customer / Supplier
4. Purchase / Sale
5. Order / Shipping
6. Return
7. InventoryMovement
8. Definir relaciones (con cardinalidades):
9. Product 1..N SKU
10. Sale 1..N SaleItem
11. Purchase 1..N PurchaseItem
12. Definir estados (máquinas de estado):
13. Order: BORRADOR → CONFIRMADO → EMPACADO → ENVIADO → ENTREGADO
14. Sale: ABIERTA → CERRADA
15. Definir acciones del negocio:
16. Confirmar pedido
17. Cerrar venta
18. Aplicar devolución

Precauciones

- Si metes “inventario” como un campo estático, luego se vuelve inconsistente.
- No mezclar “pedido” con “venta”: pedido es intención, venta es transacción.

Observaciones

- Cuando el sistema crece, cada módulo se vuelve un mini-sistema. Por eso tu dominio debe quedar limpio desde el inicio.

Salidas

- Lista formal de entidades
- Relacionamiento (ER conceptual)

Fase 3 — Modelo de datos (DB design + normalización)

Objetivo

Convertir el dominio en tablas consistentes en PostgreSQL.

Entradas

- Modelo conceptual (ER)

Pasos

1. Modelo relacional inicial:
2. tablas + PK/FK
3. Normalización a 3FN:
4. quitar redundancias
5. evitar dependencias transitivas
6. Constraints:
7. UNIQUE (username, skuCode)
8. NOT NULL donde aplique
9. Índices:
10. filtros frecuentes
11. consultas de reportes
12. Inventario por movimientos:
13. ENTRADA / SALIDA / DEVOLUCIÓN / AJUSTE
14. stock = SUM(movimientos)
15. Definir objetos extra (según necesidad):
16. índices
17. vistas
18. triggers (solo si de verdad hace falta)

Precauciones

- Triggers: úsalos solo si sabes exactamente por qué. Si el trigger “hace magia”, luego debuggear duele.
- No optimizar antes de tiempo: primero consistencia.

Observaciones

- El SQL es un contrato de verdad, pero en proyectos Nest/Prisma la “fuente de verdad” suele ser: `prisma/schema.prisma` + migraciones.

Salidas

- `database/db.sql`
- `prisma/schema.prisma`

- docs/DATABASE.md
-

Fase 4 — Contrato API (REST + DTOs + Swagger)

Objetivo

Definir cómo se comunica el frontend con el backend.

Entradas

- Módulos definidos
- Modelo de datos

Pasos

1. Diseñar endpoints “REST full” (lo típico):
2. GET/POST/PATCH
3. Agregar endpoints “acción” (cuando aplica):
4. `/sales/:id/close`
5. `/orders/:id/confirm`
6. Definir DTOs por endpoint:
7. Create/Update
8. filtros de listados
9. Definir respuesta estándar:
10. OK: `{ data: ... }`
11. Error: `{ error: { code, message } }`
12. Documentar Swagger:
13. tags por módulo
14. auth bearer

Precauciones

- Si el contrato cambia cada día, el frontend sufre.
- No mezclar endpoints “CRUD” con procesos sin estado.

Observaciones

- Swagger es tu “contrato legal técnico”: reduce discusiones.

Salidas

- docs/API.md
 - Swagger `/api/v1/docs`
-

Fase 5 — Arquitectura (módulos + capas + decisiones)

Objetivo

Evitar el “monstruo de código” (22 controllers que nadie entiende).

Entradas

- Contrato API
- Dominio

Pasos

1. Modularización por dominio (NestJS Modules).
2. Capas simples:
3. Controller (HTTP)
4. Service (reglas / orquestación)
5. Prisma (persistencia)
6. Infra común:
7. Interceptor de respuesta
8. Filtro de excepciones
9. Logger
10. Validación DTO
11. Seguridad:
12. JWT
13. RBAC

Precauciones

- “Todo en un service” mata el mantenimiento.
- Copiar/pegar sin patrones genera bugs repetitivos.

Observaciones

- Cuando un módulo crece, se divide dentro de sí: `sales/payments`, `sales/closing`, etc.

Salidas

- `docs/DECISIONS.md`
 - `docs/RBAC.md`
 - Árbol `src/` definido
-

Fase 6 — Onboarding + Scaffolding (repo profesional)

Objetivo

Que puedas clonar el repo y correrlo en 10 minutos.

Entradas

- Arquitectura definida

Pasos

1. Versiones fijas:
2. Node
3. Nest
4. Prisma
5. Repo limpio:
6. README
7. env example
8. gitignore
9. DB reproducible:
10. docker compose postgres
11. Calidad:
12. ESLint + Prettier
13. Prisma:
14. migrate
15. seed (roles + admin)
16. Scripts de operación:
17. reset db
18. init local

Precauciones

- Si no fijas versiones, el proyecto “envejece” rápido.
- Evita que el proyecto dependa de “mi PC” (Docker te salva).

Observaciones

- Esta fase es lo que te permite mantener varios proyectos freelance sin volverte loco.

Salidas

- Proyecto reproducible
- `docker compose up -d` funciona
- migraciones y seed corren

Fase 7 — Desarrollo (implementación por módulos)

Objetivo

Construir funcionalidad real, por etapas, sin caos.

Entradas

- API definida
- BD definida

Orden recomendado (V1)

1) Auth + Users/Roles 2) Catalog 3) Customers + Suppliers 4) Inventory 5) Purchases 6) Sales + Payments + Close 7) Orders + Shipping 8) Returns 9) Reports

Checklist por módulo

1. Crear Module/Controller/Service
2. Crear DTOs
3. Proteger con guards (JWT/RBAC)
4. Validar entradas (DTO + pipes)
5. Implementar errores de negocio
6. Actualizar Swagger
7. Probar el flujo manualmente

Precauciones

- No implementes reportes sin tener ventas/compras sólidas.
- No mezcles “cierre de venta” con “edición de venta”.

Observaciones

- IDE + snippets + generators te ahorran vida. El valor está en las decisiones, no en escribir 40 archivos repetidos.

Salidas

- Endpoints funcionando por módulo
-

Fase 8 — Testing (unit + integration + e2e)

Objetivo

Evitar regresiones: que una mejora no rompa ventas.

Tipos

- Unit: reglas pequeñas
- Integration: DB real
- E2E: simula cliente HTTP

Casos críticos (mínimo)

- Compra recibida → ENTRADA inventario
- Venta cerrada → SALIDA inventario
- Devolución → DEVOLUCIÓN inventario
- RBAC bloquea accesos

Precauciones

- Tests inútiles (que no cubren flujos reales) son pérdida.
- Testea lo que genera dinero y evita problemas (ventas/inventario).

Observaciones

- E2E te da confianza para refactor.

Salidas

- `test/` con casos críticos

Fase 9 — Calidad y hardening (seguridad + performance + DX)

Objetivo

Que el sistema aguante trabajo real.

Pasos

- Validación estricta
- Manejo consistente de errores
- Índices
- Paginación

- Logs útiles
- Rate limits (si aplica)

Precauciones

- “Optimizar sin medir” suele ser pérdida.

Observaciones

- El 80% de bugs reales vienen de: DTO mal validado + estados mal controlados.

Salidas

- Sistema estable
-

Fase 10 — Release/Deploy (entrega/producción)

Objetivo

Que el sistema se pueda instalar y operar.

Pasos

1. Variables de entorno producción
2. Migraciones controladas
3. Build
4. Documentación de operación

Precauciones

- Nunca deploy sin plan de rollback.

Observaciones

- Documentar “cómo arrancar” vale oro en freelance.

Salidas

- Manual de arranque
-

Fase 11 — Operación y mantenimiento

Objetivo

Mantener el sistema con mejoras graduales.

Pasos

- Logs + monitoreo
- Prioridades
- Roadmap

Precauciones

- Sin backup, el primer desastre te mata el proyecto.

Observaciones

- V2/V3 típicas:
- multi-bodega
- reservas reales
- facturación
- colas para reportes

Salidas

- Proyecto sostenible
-

Gate final V1 (checklist)\$1

Qué le falta al proyecto cuando crece (y qué significa)

Esta sección es tu mapa para entender “qué viene después” cuando el cliente dice:

“ya funciona... ahora quiero que esto sea serio y grande”.

Señales de que V1 ya quedó pequeña

- Más de 2-3 personas usando el sistema simultáneamente.
- Se pierden ventas por falta de stock real o por errores de inventario.
- Se vuelve normal vender por WhatsApp + mostrador + envíos el mismo día.
- El negocio quiere **facturación, contabilidad, análisis de ventas**.
- El cliente pide: “quiero historial de quién hizo qué y cuándo”.

Crecimiento típico por capas

1) Dominio (más reglas y más estados)

- **Cotizaciones** formales (antes de pedidos)

- **Órdenes de compra** internas (antes de compras)
- **Separación** de stock (reservado / disponible / defectuoso)
- **Promociones** y descuentos con reglas

2) Inventario más real (lo que más se complica)

- Multi-bodega (bodega principal + vitrina + sucursal)
- Lotes/seriales (ej: baterías, componentes caros)
- Productos “kit” (un producto compuesto por otros)
- Diferenciar stock “para venta” vs “para servicio/armado”

3) Reportes y analítica

- Reportes diarios automáticos
- Ranking de productos / rotación
- Margen (costo vs venta)
- Alertas de bajo stock

4) Operación y seguridad

- Auditoría (tabla de eventos o log de cambios)
- Permisos más finos (no solo roles)
- Historial de precio (precio cambia con el tiempo)

5) Arquitectura y performance

- Separar módulos internos (ej: `sales/payments`, `orders/shipping`)
- Jobs programados (cron) para tareas:
 - limpiar pedidos expirados
 - consolidar reportes
 - Cache (ej: catálogos muy consultados)

6) Integraciones

- Pasarelas de pago
- WhatsApp API (automatizar mensajes)
- Sistemas de facturación (si aplica)

Qué significa “crecer” en código

- Más carpetas no es el problema. El problema es **complejidad de reglas**.
 - Un módulo puede crecer a 30-80 archivos si incluye:
 - submódulos por proceso
 - validaciones
 - políticas de permisos
 - reportes y queries
 - eventos y jobs
-

Cómo trabajo con GitHub (criterios y flujo)

Esta sección define tu método de trabajo para que un proyecto freelance sea controlable.

Objetivo

- Tener trazabilidad de cambios.
- Poder volver atrás (rollback) sin llorar.
- Proteger la rama principal.

Flujo recomendado (solo tú o equipo pequeño)

Estrategia simple: "main estable" + "feature branches"

1. `main` siempre debe compilar y correr.
2. Cada cambio va en una rama:
3. `feature/catalog-crud`
4. `fix/sales-total-bug`
5. Merge con PR (aunque seas tú):
6. sirve como checklist mental

Commits (regla práctica)

- Commits pequeños y con mensaje claro:
- `feat: add categories endpoints`
- `fix: prevent closing sale with empty items`
- `docs: update API endpoints`

Versionado (cuando empiezas a entregar)

- Usa tags por release:
- `v1.0.0`, `v1.0.1`
- Si hay cambios grandes:
- `v1.1.0`

Issues y control de scope

- Cada pedido del cliente se vuelve un **issue**.
- Si crece el alcance, se crea un issue nuevo.

Reglas de oro

- Nunca subas `.env` real.
- Mantén seeds y migraciones dentro del repo.
- Documenta decisiones en `docs/DECISIONS.md`.

Deploy y hosting (PaaS vs VPS vs Shared Hosting)

Aquí decides dónde correr el sistema según el contexto del cliente.

Conceptos (para elegir bien)

- **PaaS (Platform as a Service):** te da deploy fácil y maneja parte del sistema.
- Ej: Railway, Render.
- **VPS (Virtual Private Server):** una máquina virtual donde tú administras todo.
- **Shared Hosting:** hosting compartido (barato), limitado para backends modernos.
- Ej: Hostinger (según plan).

Criterios para decidir

Elige PaaS cuando...

- Quieres **velocidad** y cero infraestructura.
- Proyecto pequeño/mediano.
- No quieres pelear con nginx, certificados y firewalls.
- Necesitas un demo rápido para un cliente.

Riesgos PaaS: - límites del plan - costos cambian con uso - dependencia del proveedor

Elige VPS cuando...

- Quieres control total y costos más estables.
- El cliente ya tiene hosting o quiere independencia.
- Vas a necesitar:
 - jobs
 - múltiples servicios
 - tunear performance

Riesgos VPS: - tú eres el DevOps - si no hay backups, muere el proyecto

Elige Shared Hosting cuando...

- Es un proyecto extremadamente simple (no típico NestJS).
- El cliente insiste por presupuesto.

Riesgos Shared Hosting: - límites técnicos - dolor para contenedores y procesos

Regla práctica rápida

- **Demo / V1 rápida:** PaaS
- **Negocio estable y serio:** VPS
- **Presupuesto mínimo:** shared hosting (con muchas limitaciones)

¿Qué debo asegurar en cualquier hosting?

- Backups (automáticos)

- HTTPS (certificado)
 - Variables de entorno
 - Logs
 - Migraciones controladas
-

Si voy a hacer frontend (web o desktop)

Este backend está bien, pero el frontend trae decisiones extra.

Objetivo del frontend

- Convertir casos de uso en pantallas usables.
- Reducir errores humanos (validación y flujos correctos).

Si el frontend es Web

Cosas que debes tener en cuenta

- **Auth:**
 - tokens
 - refresh (si lo implementas)
- **RBAC en UI:**
 - ocultar acciones no permitidas
 - mostrar mensajes claros
- **UX de inventario:**
 - siempre mostrar stock actual
 - bloquear ventas si stock insuficiente
- **Paginación + filtros:**
 - listas largas (productos, ventas)
- **Formularios robustos:**
 - validación cliente + servidor

Precauciones

- CORS y cookies/tokens mal gestionados = dolor.
- Si el contrato API cambia sin control, el frontend se rompe.

Observaciones

- Swagger + DTOs te facilitan generar tipos para frontend.

Si el frontend es Desktop (GUI)

Tecnologías típicas

- JavaFX (Java)
- Electron/Tauri (web tech en escritorio)

Cosas que debes tener en cuenta

- **Distribución:** instalador, versiones, actualizaciones.
- **Conectividad:** qué pasa si no hay internet.
- **Seguridad:** dónde guardas tokens (no hardcodear).
- **UI por roles:** igual que web.

Precauciones

- Desktop puede volverse más caro de mantener si no hay auto-update.
- Si el backend cambia, el desktop debe actualizarse.

Observaciones

- Desktop es útil para negocios que quieren “app cerrada”, pero web suele ser más fácil de desplegar.

Nota final (tu mentalidad correcta)

Cuando tú dices:

“lo importante es diseñar antes del código” Eso es ingeniería real.

El IDE, los generators y los scaffolds te ahorran el trabajo repetitivo. Tu valor profesional está en: - entender el dominio - proteger el inventario - mantener el historial - evitar bugs de operación - entregar un sistema reproducible